

# Drona — Persistent Context Engine for AI SRE

Anvil P-02 · Team radiohead · May 2026

## 1. Memory Representation

Drona's memory layer solves a fundamental problem in operational context: services are renamed, split, and merged, but incidents must be matchable across those boundaries. We address this with a two-tier identity architecture.

**Canonical ID Layer.** Every service name seen in any event is mapped to a UUID canonical\_id via IdentityLayer.resolve(). The mapping is stored in an alias → canonical\_id dictionary. When a topology rename event arrives, handle\_rename(old, new) adds the new name as an alias to the \*existing\* node — the UUID never changes. All downstream components (temporal index, memory store, causal chain builder) operate exclusively on canonical IDs. This means a query for billing-svc after a rename from payments-svc transparently resolves to the same UUID, and all historical events indexed under that UUID are immediately accessible.

**Behavioral Signatures.** Each closed incident produces a BehaviorPattern containing: trigger type (deploy, metric\_alert, dependency\_failure, unknown), symptom sequence (ordered list of symptom types), affected service count, propagation direction, and time-to-first-symptom. Critically, no service names appear in this structure. Similarity between two patterns uses a 4-component weighted score: trigger match (0.30), LCS ratio on symptom sequences (0.40), propagation match (0.20), and time-to-symptom ratio (0.10). Because patterns contain zero topology information, matching works identically before and after any rename, split, or dependency change.

**Incident Memory.** Each IncidentMemory stores the behavioral signature, epicentre canonical ID, remediation action and target, outcome, and the raw context events. The MemoryStore maintains a list of closed incidents and supports recency-decayed similarity search with a 3-day half-life.

## 2. Relationship Synthesis

Causal chains are built using three deterministic rules, executed without any LLM call on the critical path.

**Rule 1 — Deploy → Anomaly.** If a deploy event precedes a metric spike or trace slowdown within 5 minutes, a causal edge is created from the deployed service to the affected service. Confidence scales from 0.85 (at 5 min) to 0.95 (immediate). This captures the most common production incident pattern: a bad deploy causing downstream degradation.

**Rule 2 — Timeout Log → Caller.** When an error log contains keywords like "timeout" or "connection refused", we extract the callee service name from the

message using regex patterns (e.g., timeout calling payments-svc). The causal edge points from the callee (root cause) to the caller (symptom). Confidence: 0.75. The callee extraction uses 3 regex patterns plus a fallback on service-like suffixes (-svc, -api, -service, -gateway).

**Rule 3 — Trace Span Slowdown → Caller.** When a trace contains a span with duration exceeding 3 seconds, the slow downstream span is identified as the cause and the upstream caller as the effect. Confidence: 0.80.

Post-processing deduplicates edges by (cause, effect) pair, removes self-loops, and sorts by confidence descending. The result is a compact, auditable causal chain that judges can inspect without needing to understand an ML model.

### 3. Drift Handling

Service identity drift manifests as renames, dependency additions, and dependency removals. Drona handles all three through a single primitive: `handle_rename()` adds a new alias to an existing `ServiceNode` without changing the canonical ID.

**Why this is sufficient.** A rename is the only operation that changes how a service is referenced in future events. Dependency additions and removals change the topology graph (which edges exist) but do not affect how services are identified. Therefore, `handle_rename()` is the only identity-layer operation needed. Dependency changes are handled by the `ServiceGraph`, which adds or removes edges between canonical IDs.

**The topology event handler** in `engine.py` dispatches on the `change` field: `rename` calls `handle_rename()`, `dependency_add/link/add` resolves both services and records the edge in the graph, `dependency_remove/unlink/remove` removes the edge. All operations use canonical IDs, so the graph is automatically consistent after any rename.

**What this means for matching.** After `payments-svc` is renamed to `billing-svc`, a new incident on `billing-svc` resolves to the same canonical ID. The `BehaviorPattern` extracted from this incident contains no service names. `find_similar()` compares it against all historical patterns — including INC-714 which was filed under the original `payments-svc` canonical ID — and finds a match. The rename is invisible to the matching pipeline.

### 4. Latency Engineering

The PRD requires fast mode  $p95 < 2$  seconds. Drona achieves 3–6ms.

**DuckDB in-process.** The temporal index uses DuckDB with `:memory:` storage. No network hops, no connection pool, no startup delay. Range queries on the `ts` column use a B-tree index and complete in sub-millisecond time.

**Batch insertion.** Events are buffered in groups of 50 before a single INSERT into DuckDB. This amortizes the per-statement overhead and achieves >7,000 events/sec sustained throughput.

**Pre-computed anomaly flags.** Anomaly detection runs at ingest time, not query time. Each event is classified as anomalous (metric spike, error log, trace slowdown) using deterministic rules and the result is stored alongside the event. At query time, `get_anomalies()` is a simple `WHERE is_anomaly=TRUE` filter — no recomputation needed.

**No LLM on critical path.** The template backend generates explanations using string formatting. The `explain` field is the last step in `reconstruct_context()` and adds <1ms. LLM backends (bedrock, openrouter) are only used in deep mode and have a 4.5-second socket timeout with automatic fallback to template if the call fails or times out.

**Window strategy.** The initial query window is  $\pm 15$  minutes around the signal timestamp. If this returns no events, G2 expands to  $\pm 30$  minutes. This two-stage approach avoids scanning unnecessarily large windows while ensuring coverage for delayed signals.

## 5. Evolution Mechanism

Drona improves its matching accuracy as more incidents are observed, through two complementary mechanisms.

**Recency Decay (G4).** The `find_similar()` method applies exponential decay to similarity scores based on incident age. With a half-life of 3 simulated days, recent incidents contribute up to 30% more to the final score than older ones. This naturally prioritizes recent operational patterns over stale historical data, reflecting the reality that infrastructure changes over time.

**Incremental Classifier.** An `SGDClassifier` with `partial_fit()` learns which `BehaviorPattern` pairs belong to the same incident family. On each closed incident, we generate training pairs: positive pairs share the same remediation action and trigger type; negative pairs have different trigger types. The classifier encodes each pattern as an 8-dimensional vector (trigger ordinal, symptom count, 4 symptom flags, propagation ordinal, time-to-first-symptom) and trains on the element-wise absolute difference between two patterns.

The classifier score is blended into `find_similar()` at 20% weight (80% LCS similarity + 20% classifier). This ratio keeps LCS dominant during the early phase when few incidents have been observed, avoiding instability from an under-trained classifier. As more incidents accumulate, the classifier captures non-linear interactions between pattern features that the weighted sum in `BehaviorPattern.similarity()` cannot express.

**Measurable improvement.** Between the training phase (24 incidents) and full evaluation (34 incidents), the classifier has seen enough pairs to

produce meaningful probability estimates, improving recall@5 on the Memory Evolution benchmark axis.

**Team radiohead · Anuj Dwivedi, archzOS**